

Writing Quality Attributes (NFRs)

A student guide for recording **app-wide quality bars** ("ilities") in `docs/nfr/quality-attributes.md` : what they are, how to fill each column, how to set **Status**, and when to use a feature FR/SC instead.

Living table: [quality-attributes.md](#)

Folder index: [README.md](#)

Agent literacy: [.cursor/rules/quality-attributes.mdc](#)

Related: [writing ADRs](#) · [writing feature requirements](#) · [writing feature design](#)

What quality attributes are

Quality attributes (non-functional requirements, or NFRs) describe **how good** the system should be — security posture, performance, observability, maintainability — not **what feature** to build next.

Artifact	Question
Feature specs	<i>What</i> must the product do?
Quality attributes (<code>docs/nfr/</code>)	<i>How good</i> must it be across the product?
ADRs	<i>Why</i> this Approach for a quality?
Cursor rules	<i>How</i> must code meet an Accepted bar day to day?

NFRs here do **not** authorize new product behavior by themselves. A **Deferred** performance number does not mean "add caching this sprint" unless a feature **FR-00N** / **SC-00N**, Gherkin, or an instructor explicitly requires it.

When to edit the NFR table (vs a feature or ADR)

Situation	Write / update...
App-wide bar or explicit non-goal	quality-attributes.md row
<i>How</i> to meet a bar (auth model, DB, logging approach)	ADR — then link it from the row
Ongoing coding constraint for agents	Cursor rule — then link it from the row
Quality that only one feature needs	Feature FR-00N / SC-00N (+ Gherkin); optional one-line pointer in the table
New user-facing capability	Feature spec — not a new NFR row alone

Rule of thumb: if every feature must respect the bar (or we must explicitly refuse to build for it), it belongs in the quality-attributes table.

Examples from this project

Attribute	Status	Why it belongs in NFRs
Security	Accepted	Cross-cutting: auth on protected routes, 0 cross-user leaks, 404 not 403

Data integrity	Accepted	Every list/todo row has owning <code>userId</code> — app-wide invariant
Observability	Accepted (minimal)	Thin Winston logging bar — meet Approach, don't invent a full APM platform
Performance	Deferred	Illustrative p95 / first-paint numbers — not a CI gate yet
Availability / Scalability / i18n	Out of scope	Explicit non-goals (no multi-region HA, no i18n framework)
Maintainability	Accepted	Specs + Test Coverage Map + <code>npm test</code> as process bar

The table shape

Every product keeps one main table in [quality-attributes.md](#):

Attribute	Target	Approach	How we verify	Status	Links
Security	...	...	...	Accepted	ADR, rules, ...

Column principles

Column	Write this	Avoid
Attribute	Standard ility name (Security, Performance, ...)	Vague "Quality" or feature names ("Lists")
Target	Prefer a number or countable bar (100%, 0, p95 < 200 ms, ≤ 2 clicks)	"Make it fast" with no measure
Approach	How the bar is realized <i>or</i> why it is limited / out of scope	A second Target paragraph
How we verify	Tests, manual check, or N/A	"Somehow"
Status	One of the four values below	Mixing Accepted wording with Deferred intent
Links	ADR / rule / code path / feature pointer, or —	Orphan Accepted rows with no enforcement link

Teaching note: Targets on **Deferred** / **Out of scope** rows are often **illustrative classroom examples**, not production SLOs. Only **Accepted** (and feature FR/SC) constrain implementation by default.

Status values — principles

Status	Meaning	When writing / coding
Accepted	In force for this product	Do not regress; back with Links (ADR/rule/tests)
Accepted (minimal)	Thin bar in force	Meet the stated Approach only; do not expand into a platform

Deferred	Documented backlog / learning example	Guidance only — implement only if a feature or human requires it
Out of scope	Explicit non-goal	Do not invent HA, i18n frameworks, etc.

Choosing Status

1. **Can you verify it today** (automated or documented manual gate)? → candidate for **Accepted** or **Accepted (minimal)**.
2. **Useful to teach or plan, but not enforced?** → **Deferred** (keep a numeric Target as an example).
3. **We will not build for this?** → **Out of scope** (say so in Approach).
4. **Promoting Deferred → Accepted** requires matching **How we verify** — do not flip Status without tests or a real manual gate.

Links column — principles

Link type	Use when
ADR (<i>docs/adr/...</i>)	<i>Why</i> this Approach (e.g. ADR-0002 for Security)
Cursor rule (<i>.cursor/rules/...</i>)	<i>How</i> agents must behave every day
Code path	Minimal concrete implementation (e.g. <code>logger.js</code>)
Feature / framework	Process or FR/SC that carries the bar
—	Common for Deferred rows with no artifact yet

When Approach changes → update or add an ADR and refresh **Links**.

When agents need a lasting constraint → add/update a Cursor rule and link it.

Principles for writing good NFR rows

1. **One ility per row**. Don't combine Security and Performance into one Attribute.
2. **Prefer measurable Targets**. Numbers beat adjectives.
3. **Separate Target from Approach**. Target = bar; Approach = how/limits.
4. **Be honest about Status**. Deferred numbers are not secret Accepted gates.
5. **Link enforcement for Accepted rows**. An Accepted Security row without ADR/rule/tests is a smell.
6. **Keep feature-local quality in features**. Validation copy, one-screen UX notes → FR/SC + Gherkin.
7. **Out of scope is a feature, not an omission**. Explicit non-goals stop agents from inventing multi-region HA or i18n.
8. **Accepted (minimal) means thin**. Observability via Winston logs ≠ "build Datadog."
9. **Align with ADRs and rules**. Don't contradict ADR-0002 in the Security Approach.
10. **Update the table when reality changes**. New auth model, new logging, or a promoted bar → edit the row in the same change set when possible.

Feature-local NFRs

If only one feature needs a bar:

1. Put it under that feature's **Requirements (FR-00N)** and/or **Success Criteria (SC-00N)**.
2. Add Gherkin when tests must prove it.
3. Optionally add a one-line pointer on a related app-wide row ("see Feature N **FR-00N** / **SC-00N**").

Do **not** invent a new feature file solely to "add performance."

Workflow: add or change a bar

1. Edit [quality-attributes.md](#) — **Target, Approach, How we verify, Status, Links**. Prefer a number in **Target**.
 2. If **Approach** changes → [write/update an ADR](#); put it in **Links**.
 3. If agents need a lasting coding constraint → Cursor rule; put it in **Links**.
 4. If API/schema/rules change as a result → `features/reference/` in the same feature PR.
 5. **Deferred** → **Accepted** only with verification that matches **How we verify**.
-

Checklist (before you call a row “done”)

- ☐ Attribute name is a clear ility (not a feature title)
 - ☐ Target is measurable (or explicitly qualitative with a clear bar)
 - ☐ Approach explains realization **or** deliberate limit / non-goal
 - ☐ How we verify is realistic for the Status
 - ☐ Status matches enforcement reality (Accepted ↔ Links / tests)
 - ☐ Links point at ADR / rule / code / feature — or — on purpose
 - ☐ No conflict with existing Accepted ADRs or Cursor rules
 - ☐ Feature-only concerns left in FR/SC, not forced into the app-wide table
-

Anti-patterns

Avoid	Do instead
Treating every Deferred Target as a sprint commitment	Honor Status; only Accepted (+ feature FR/SC) constrain by default
“Make it secure” with no Target	100% protected routes auth'd; 0 cross-user leaks in tests
Accepted row with Links = — forever	Add ADR/rule/tests or demote to Deferred
Putting “user can create a list” in NFRs	Feature story / FR
Inventing i18n or HA because a Target number looks cool	Respect Out of scope
Duplicating the whole NFR table inside every feature	Link here; keep feature-local bars in FR/SC

How this fits the other guides

Product behavior (what)	→ <code>features/writing-feature-requirements.md</code> <code>features/writing-feature-design.md</code>
Architecture why	→ <code>docs/adr/writing-adrs.md</code>
Quality bars (how good)	→ <code>docs/nfr/writing-quality-attributes.md</code> (this file)
Day-to-day how for agents	→ <code>.cursor/rules/*.mdc</code> (incl. <code>quality-attributes.mdc</code>)